Question: 1

[1] explain characteristics or Feature of SE.

Ans: The term software engineering is the product of two words, software, and engineering.

- The software is a collection of integrated programs.
- Engineering is the application of scientific and practical knowledge to invent, design, build, maintain, and improve frameworks, processes, etc.
- Software engineering is process of designing, developing, testing and maintaining software.
- There are 4 attribute in se:
  1. Efficiency
  2. Reliability
  3. Reusability
  4. Maintainability

Characteristics of the Software

- It is intangible, meaning it cannot be seen or touched.
- It is non-perishable, meaning it does not degrade over time.
- It is easy to replicate, meaning it can be copied and distributed easily.
- It can be complex.
- It can be difficult to understand and modify for the system.
- It can be affected by changing requirements.
- It can be impacted by bugs and other issues.
- Good programming abilities.
- Good communication skills.
- High motivation.
- Intelligence.

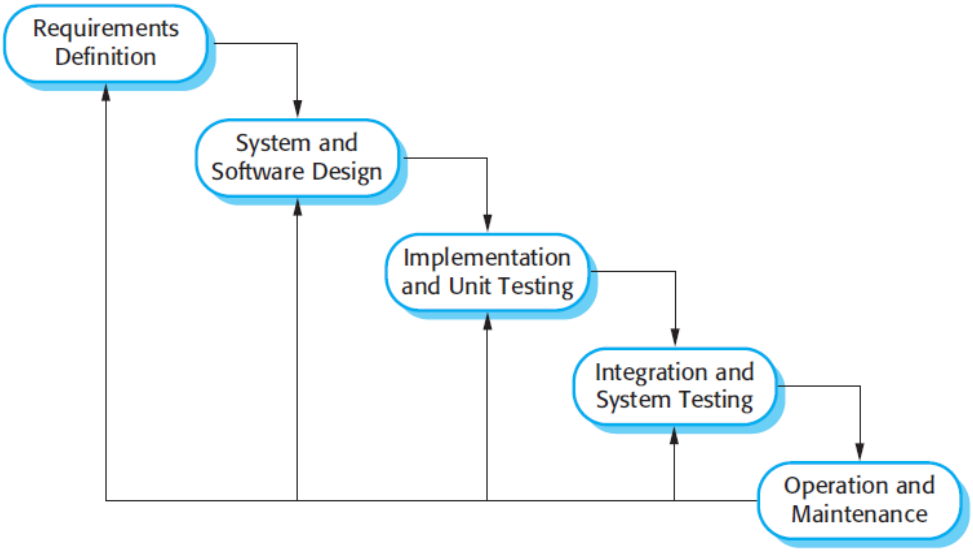
[2] explain waterfall model in detail

Ans: Software Process: a software process is set of activities that development team performs to create a software program.

- Waterfall Model: The waterfall model is a software development model used in the context of large, complex projects, typically in the field of information technology.
- The waterfall model is useful in situations where the project requirements

are well-defined and the project goals are clear.

- The phases of the waterfall model are: Requirements, Design, Implementation, Testing, and Maintenance.



1. Requirements analysis and specification phase: the first phase involves requirements regarding the software are gathered from the customer and then the gathered requirements are analyzed. The goal of the analysis part is to remove incompleteness and inconsistencies.

   Requirement specification: These analyzed requirements are documented in a software requirement specification (SRS) document. SRS document serves as a contract between the development team and customers.

2. Design Phase: once the requirement are understood, the design phase begins. This involves creating a detailed design document that outlines the software architecture, user interface and system components.

3. Implementation and unit testing: the implementation phase involves coding the software based on the design specifications.
   - This phase also includes unit testing to ensure that each component of the software is working as excepted.
4. Integration and System Testing: This phase is highly crucial as the quality of the end product is determined by the cogency of the testing carried out.
   - The better output will lead to satisfied customers, lower maintenance costs and accurate results.

5. Operation and maintenance phase: this phase involves maintaining and updating the software after it has been delivered to the customer, installed, and operational.

[3] explain advantage and disadvantage in waterfall model.

Ans: Software Process: a software process is set of activities that development team performs to create a software program.

- Waterfall Model: The waterfall model is a software development model used in the context of large, complex projects, typically in the field of information technology.
- The waterfall model is useful in situations where the project requirements are well-defined and the project goals are clear.
- The phases of the waterfall model are: Requirements, Design, Implementation, Testing, and Maintenance.

- Advantage:

- Simple and easy to understand and use.
- Work well for smaller projects where requirements are very well understood.
- Well understood milestone
- Easy to arrange tasks
- Easy to manage due to flexibility of the model.

- Disadvantage:

- High amount of risk.
- Not a good model for complex and object-oriented projects.
- Poor model of long and ongoing projects.
- It is difficult to measure progress within stages.
- It cannot changing requirements
- Adjusting scope during the life cycle can end a project

[4] Explain integration testing in detail or Define the term top up, bottom up and sandwich testing in detail.

Ans:

Integration testing: Integration testing is a software testing technique that focuses on the interactions and data exchange between different components or modules of a software application.

- It occurs after unit testing and before system testing.
- The goal of integration testing is to identify any problems or bugs that arise when different components are combined and interact with each other.

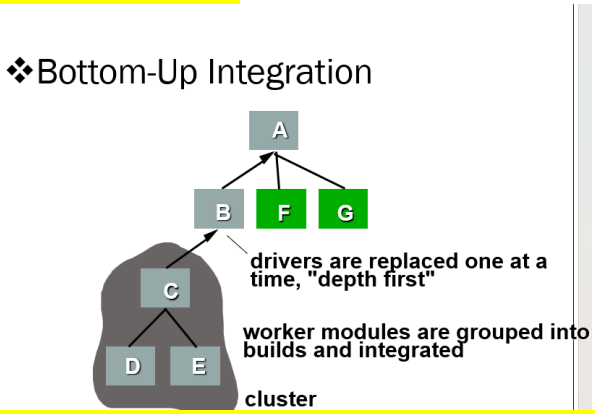
- There are four types of integration testing approaches.

a. Big-Bang integration testing
b. Bottom-up integration testing
c. Top-down integration testing
d. Mixed integration testing

1. Big-Bang Integration Testing: Big-bang integration testing is a type of integration testing that combines all the modules or components of a system into a single unit.

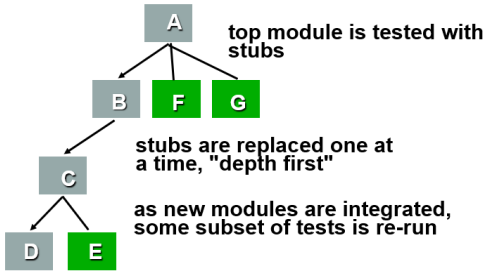
2. Bottom-Up Integration Testing: In bottom-up testing, each module at lower levels are tested with higher modules until all modules are tested.

   - This integration testing uses test drivers to drive and pass proper data to the lower-level modules.



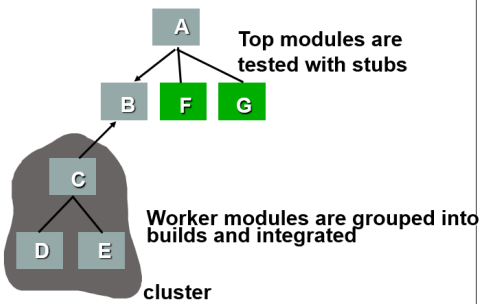
3. Top-Down Integration Testing: In this integration testing, testing takes place from top to bottom. First, high-level modules are tested and then low-level modules and finally integrating the low-level modules to a high level to ensure the system is working as intended.

❖Top Down Integration



4. Mixed Integration Testing: A mixed integration testing is also called sandwiched integration testing.
   - A mixed integration testing follows a combination of top down and bottom-up testing approaches.
   - In top-down approach, testing can start only after the top-level module have been coded and unit tested.
   - In bottom-up approach, testing can start only after the bottom level modules are ready.

❖Sandwich Testing



[5] explain design principle and method in detail

Ans: The software design process is a systematic approach to creating a blueprint for the software.

Software Design is also a process to plan or convert the software requirements into a step that are needed to be carried out to develop a software system. There are several principles that are used to organize and arrange the structural components of Software design.

- It translates user requirements into a structured design that developers can implement.
- The goal is reliable, maintainable, and scalable software that meets objectives.
- Principles of good software design guide developers in creating high-quality, maintainable, and efficient software solutions.

- Principles Of Software Design :

1. Should not suffer from “Tunnel Vision” –

While designing the process, it should not suffer from “tunnel vision” which means that is should not only focus on completing or achieving the aim but on other effects also.

2. Traceable to analysis model –

The design process should be traceable to the analysis model which means it should satisfy all the requirements that software requires to develop a high-quality product.

3. Should not “Reinvent The Wheel” –

The design process should not reinvent the wheel that means it should not waste time or effort in creating things that already exist. Due to this, the overall development will get increased.

4. Minimize Intellectual distance –

The design process should reduce the gap between real-world problems and software solutions for that problem meaning it should simply minimize intellectual distance.

5. Exhibit uniformity and integration –

The design should display uniformity which means it should be uniform throughout the process without any change. Integration means it should mix or combine all parts of software i.e. subsystems into one system.

6. Accommodate change –

The software should be designed in such a way that it accommodates the change implying that the software should adjust to the change that is required to be done as per the user’s need.

7. Degrade gently –

The software should be designed in such a way that it degrades gracefully which means it should work properly even if an error occurs during the execution.

8. Assessed or quality –

The design should be assessed or evaluated for the quality meaning that during the evaluation, the quality of the design needs to be checked and focused on.

9. Review to discover errors –
The design should be reviewed which means that the overall evaluation should be done to check if there is any error present or if it can be minimized.

10. Design is not coding and coding is not design –
Design means describing the logic of the program to solve any problem and coding is a type of language that is used for the implementation of a design.

[6] Define the term validation and verification in S.E.

Ans:

1. Verification is the process to ensure whether the product that is developed is right or not.
   - Verification Testing is known as Static Testing.
   - Verification testing Method involves review, inspection, unit testing and integration testing.
   - It is static and dynamic activities.
   - Here are some of the activities that are involved in verification.
     - Inspections
     - Reviews
     - Walkthroughs
     - Desk-checking

2. Validation is the process of checking whether I had done the right thing or not.
   - It is the process of checking the validation of product.
   - It is the process check the final product against specification.
   - Validation Testing is known as Dynamic Testing.
   - Aim is to made final product error free.
   - Method involves system testing.
   - It is only dynamic activities.
   - Here are some of the activities that are involved in Validation.
     1. Black Box Testing
     2. White Box Testing
     3. Unit Testing
     4. Integration Testing

[7] Explain specialized Process model in detail.

Ans: Software Process: a software process is set of activities that development team performs to create a software program.

- A software process model is an abstraction of the actual process, which is being described.
- It can also be defined as a simplified representation of a software process.
  - Basic software process models on which different type of software process models can be implemented:

1. A workflow Model –
   It is the sequential series of tasks and decisions that make up a business process.

2. The Waterfall Model –
   The waterfall model is a software development model used in the context of large, complex projects, typically in the field of information technology.
   Phases in waterfall model:
   (i) Requirements
   (ii) Design
   (iii) Implementation
   (iv) Testing
   (v) Maintenance.

3. Dataflow Model –
   It is diagrammatic representation of the flow and exchange of information within a system.

4. Evolutionary Development Model –
   Following activities are considered in this method:
   - (i) Specification
   - (ii) Development
   - (iii) Validation

5. Role / Action Model –
   Roles of the people involved in the software process and the activities.

[8] explain incremental process model in detail.

Ans: Incremental Process Model is also known as the Successive version model. This article focuses on discussing the Incremental Process Model in detail.

- Phases of incremental model:
  1. Requirement analysis: In the first phase of the incremental model, the product analysis expertise identifies the requirements.
- the system functional requirements are understood by the requirement analysis team. To develop the software under the incremental model, this phase

performs a crucial role.

2. Design & Development: In this phase of the Incremental model of SDLC, the design of the system functionality and the development method are finished with success.

- When software develops new practicality, the incremental model uses style and development phase.

3. Testing: In the incremental model, the testing phase checks the performance of each existing function as well as additional functionality.

- In the testing phase, the various methods are used to test the behavior of each task.

4. Implementation: Implementation phase enables the coding phase of the development system.

- It involves the final coding that design in the designing and development phase and tests the functionality in the testing phase.
- After completion of this phase, the number of the product working is enhanced and upgraded up to the final system product

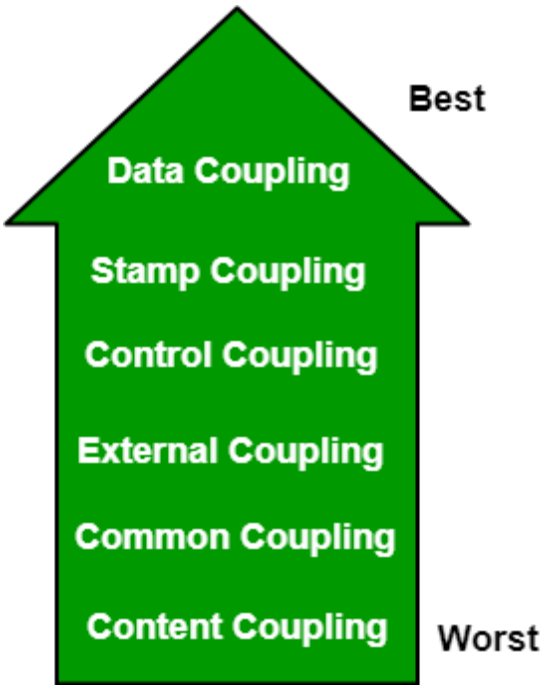
- Advantage of Incremental Model:
  - Easier to test and debug
  - More flexible.
  - Simple to manage risk
- Disadvantage of Incremental Model
  - Need for good planning
  - Total Cost is high.

[9] explain Cohesion and coupling.

Ans:

1. Coupling: we represent coupling in top level design and its describe the inter-dependency and intersection between the module of software.

- Coupling should be low for better software design



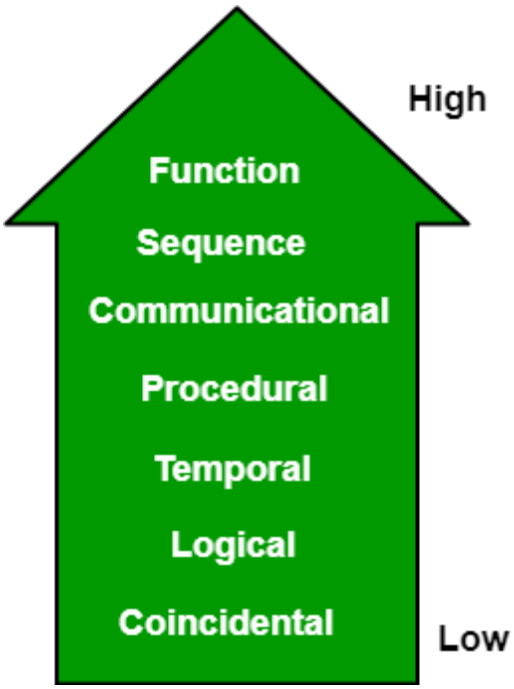
- Types of coupling:

i. Content coupling: it is a bed coupling because it's coupling should be very high.
ii. Common coupling: same global data share between two module. It is also know as global coupling
iii. External coupling: share two module between the external data format and communication protocol it called external coupling.
iv. Control coupling: this coupling, one module control another module it called control coupling.
v. Stamp coupling: the many module share to the same type data it is know as stamp coupling.
vi. Data Coupling: When data of one module is passed to another module, this is called data coupling.

- It is vary best coupling because it should be low for better software design

2. Cohesion: cohesion represent detail design and it describe how the element in a particular module are closely related to each other and it should be high for better software design.

- Cohesion is an ordinal type of measurement and is generally described as "high cohesion" or "low cohesion."



- Types of cohesion:

1. Functional cohesion: it is better cohesion. It is very high
- A functional cohesion performs the task and functions. It is an ideal situation

2. Sequential cohesion: where the output from one component of the sequence is input to the next. It is know as sequential cohesion
3. Communication cohesion: A module is said to have communicational cohesion, if all tasks of the module refer to or update the same data structure,
4. Procedural cohesion: A module is said to be procedural cohesion if the set of purpose of the module are all parts of a procedure in which particular sequence of steps has to be carried out for achieving a goal
5. Temporal Cohesion: When a module includes functions that are associated by the fact that all the methods must be executed in the same time, the module is said to exhibit temporal cohesion.
6. Logical Cohesion: A module is said to be logically cohesive if all the elements of the module perform a similar operation. For example Error handling, data input and data output, etc.
7. Coincidental Cohesion: A module is said to have coincidental cohesion if it performs a set of tasks that are associated with each other very loosely, if at all.

Question: 2

[1] Explain the Software Reliability in detail.

Ans: Software Reliability means Operational reliability. It is described as the ability of a system or component to perform its required functions under static conditions.

- Software Reliability is an essential connect of software quality, composed with functionality, usability, capability, maintainability, and documentation.

- It is measured per some unit of time.
- Software Reliability starts with many faults in the system when first created.
- After testing and debugging enter a useful life cycle.
- Useful life includes upgrades made to the system which bring about new faults.
- The system needs to then be tested to reduce faults.

- There are five key component in software reliability

3. Failure: Failures can be caused by defects, errors, faults, or external factors
4. Fault: A fault, also known as a defect or error, is an underlying problem. Faults can manifest as coding errors, design flaws.
5. Reliability: Reliability refers to the ability of the software system to perform its functions consistently and predictably over time without failures, errors, or downtime.
6. Mean Time Between Failures (MTBF): It represents the average time elapsed between consecutive failures during normal operation.
7. Fault Tolerance: Fault-tolerant systems are designed to detect, isolate, and recover from failures gracefully, minimizing their impact on system availability and user experience.

- Several factors influence the reliability of software systems, including:

8. Quality of Design: Well-designed software with clear requirements, robust architecture, and thorough testing is more likely to be reliable.

9. Quality of Implementation: The quality of software implementation, including coding standards, programming practices, and testing techniques, affects the likelihood of defects and errors.

10. Environmental Factors: The operating environment in which the software operates, including hardware platform, software dependencies, network infrastructure, and user interactions, can influence reliability.

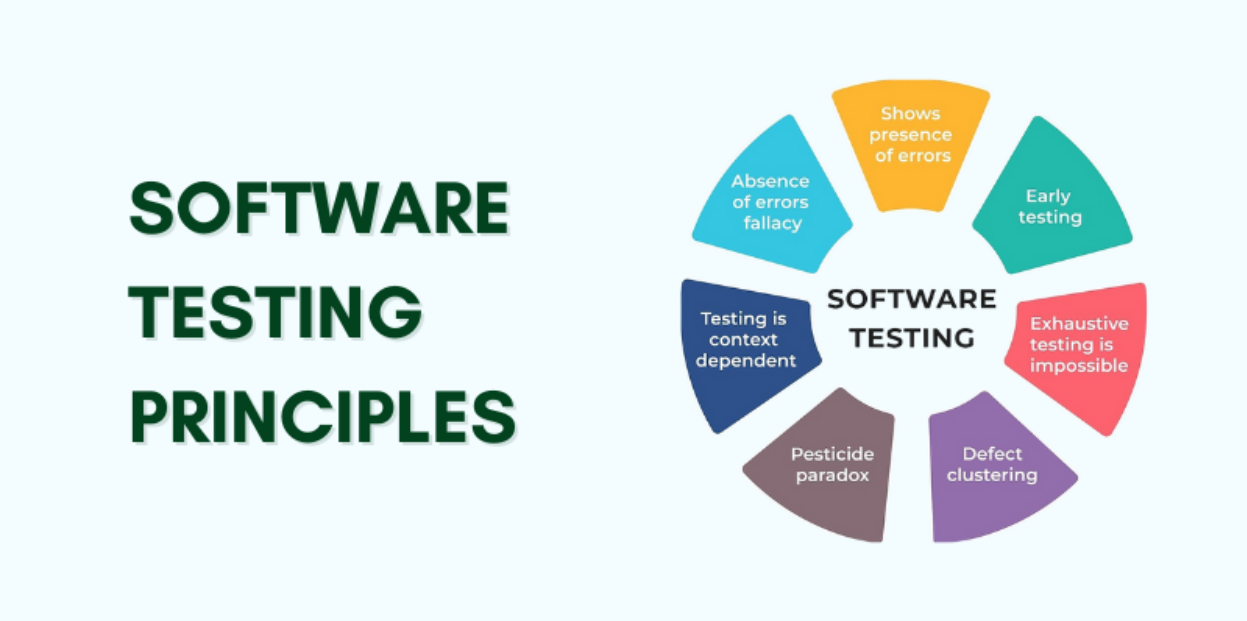
11. Usage Patterns: The frequency and intensity of software usage, as well as the diversity of user interactions and input data, can impact reliability.

[2] Explain Testing Principal in detail.

Ans: Software testing is a procedure of implementing software

- Software testing is the application to identify the defects or bugs.
- the seven different testing principles:

1. Testing shows the presence of defects
2. Early Testing
3. Exhaustive Testing is not possible
4. Defect Clustering
5. Pesticide Paradox
6. Testing is context-dependent
7. Absence of errors fallacy



1. Testing shows the presence of defects: The test engineer will test the application to make sure that the application is bug or defects free.
   - While doing testing, we can only identify that the application or software has any errors.
   - the primary purpose of testing is to identify the numbers of unknown bugs with the help of various methods and testing techniques.

2. Early Testing: Here early testing means that all the testing activities should start in the early stages of the software development life cycle.
   - so that requirement analysis stage to identify the defects because if we find the bugs at an early stage, it will be fixed in the initial stage itself.

3. Exhaustive Testing is not possible: As exhaustive testing is not possible, we need an optimal amount of testing based on the application's risk evaluation.
   - If the software will test every test case then it will take more cost, effort, etc., which is impractical.

4. Defect clustering: Defect Clustering which states that a small number of modules contain most of the defects detected.

- This is the application of the Pareto Principle to software testing, which states that we can identify that approx. 80% of the problems are found in 20% of the modules.

5. Pesticide paradox: these pesticide paradoxes, it is very significant to review all the test cases frequently.
   - Repeating the same test cases, again and again, will not find new bugs. So it is necessary to review the test cases and add or update test cases to find new bugs.

6. Testing is context-dependent: The testing approach depends on the context of the software developed. Different types of software need to perform different types of testing. For example, The testing of the e-commerce site is different from the testing of the Android application.

7. Absence of errors fallacy: The absence of detected defects does not proof that the software is error-free

   - the application is completely tested and there are no bugs identified before the release, so we can say that the application is 99 percent bug-free. But there is the chance when the application is tested beside the incorrect requirements
   - The absence of error fallacy means identifying and fixing the bugs would not help if the application is impractical.

[3] What is Risk? Explain categories of Risk in S.E.

Ans: "Tomorrow problems are today's risk." Hence, a clear definition of a "risk".

- Risk simply represents the possibility of loss and injury.
- These potential issues might harm cost or technical success of the project.

There are five type of risks:

1. Schedule Risk: Schedule related risks refers to time related risks or project delivery related planning risks. The wrong schedule affects the project development and delivery.

- Some reasons for Schedule risks –
  - Time is not estimated perfectly
  - Improper resource allocation

2. Budget Risk: Budget related risks refers to the monetary risks mainly it occurs due to budget overruns.

- Some reasons for Budget risks –
  - Wrong/Improper budget estimation
  - Mismanagement in budget handling
  - Improper tracking of Budget

3. Operational Risks: Operational risk refers to the procedural risks means these are the risks which happen in day-to-day operational activities during project development due some external operational risks.

Some reasons for Operational risks –

- Improper management of tasks
- No proper planning about project

4. Technical Risks: Technical risks refers to the functional risk or performance risk which means this technical risk mainly associated with functionality of product.

Some reasons for Technical risks –

- Less use of future technologies

5. Programmatic Risks: Programmatic risks refers to the external risk. These risks come from outside and it is out of control of programs.

Some reasons for Programmatic risks –

- Changes in Government rules/policy

- Loss of contracts due to any reason

[4] Explain QA vs QC.

Ans:

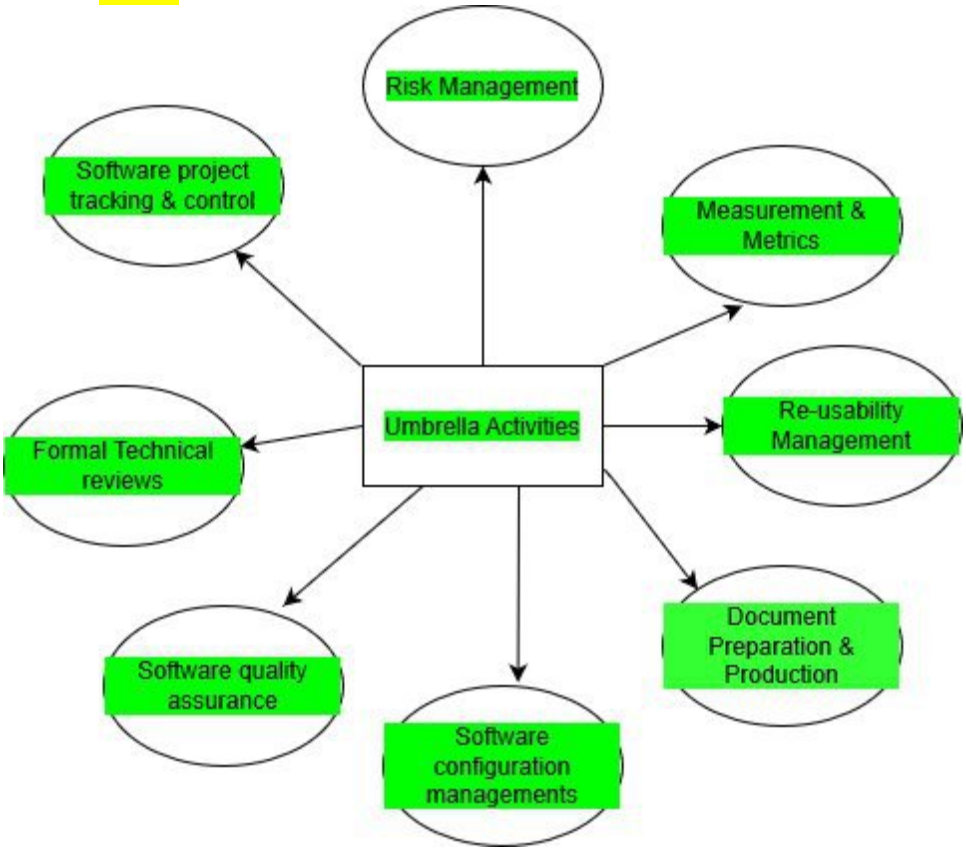
Parameters	Quality Assurance (QA)	Quality Control (QC)
Quality request	It focuses on providing assurance that the quality requested will be achieved.	It focuses on fulfilling the quality requested.
Technique	It is the technique of managing quality.	It is the technique to verify quality.
Involved	It is involved during the development phase.	It is not included during the development phase.
Execution program	It does not include the execution of the program.	It always includes the execution of the program.
Type of tool	It is a managerial tool.	It is a corrective tool.
Process/ Product-oriented	It is process oriented.	It is product oriented.
Aim	The aim of quality assurance is to prevent defects.	The aim of quality control is to identify and improve the defects.
Perform	It is performed before Quality Control.	It is performed after the Quality Assurance activity is done.
Technique type	It is a preventive technique.	It is a corrective technique.
Measure type	It is a proactive measure.	It is a reactive measure.
SDLC/ STLC?	It is responsible for the entire software development life cycle.	It is responsible for the software testing life cycle.
Activity level	QA is a low-level activity	It is a high-level
Focus	Pays main focus is on the intermediate process.	Its primary focus is on final products.
Team	All team members of the project are involved.	Generally, the testing team of the project is involved.

Aim	It aims to prevent defects in the system.	It aims to identify defects or bugs in the system.
Time consumption	It is a less time-consuming activity.	It is a more time-consuming activity.
Which statistical technique was applied?	Statistical Process Control (SPC) statistical technique is applied to Quality Assurance.	Statistical Quality Control (SQC) statistical technique is applied to Quality Control.
Example	Verification	Validation

[5] explain Umbrella Activities in se.

Ans: Software engineering is a collection of interconnected phases. These steps are expressed or available in different ways in different software process models.

- Umbrella activities are those activities to be performed through the entire software process.
- Umbrella activities are a series of steps followed by a software development team.



1. Software project tracking and control: This activity allows the software team

to access progress against the project plan and take any necessary action to maintain the schedule.

2. Risk management: Risk management accesses that may affect the outcome project or the quality of the product.

3. Software quality assurance: this defines and conducts the activities required to ensure software quality.
- The software quality is checked, such as user experience, functionality after complete development.

4. Technical reviews: It assesses software engineering work products in an effort to uncover and remove errors before they are propagated to the next activity.

5. Measurement: This includes all measurements of all aspects of the software project.

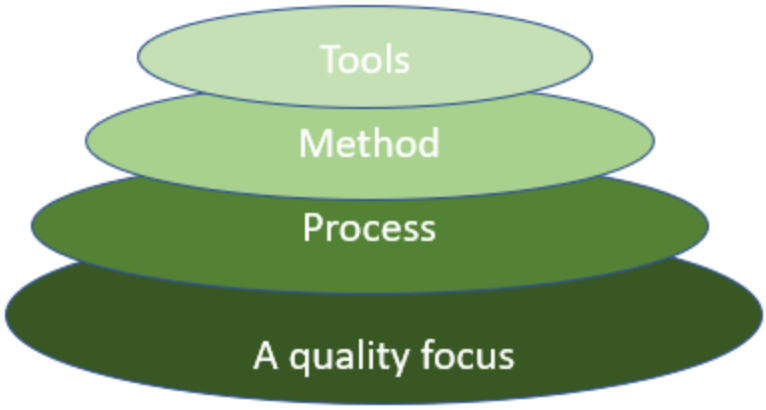
6. Software configuration management: It manages the impact of changes throughout the software development process.

7. Reusability management: This includes the approval of any part of a backing-up software project.

[6] justify layer technology

Ans: Software engineering is a fully layered technology, to develop software we need to go from one layer to another.

- All the layers are connected and each layer demands the fulfillment of the previous layer



- Layered technology is divided into four parts:

1. A quality focus: Main principle of software engineering is quality focus.

- It defines the continuous process improvement.
- It also focuses on maintainability and usability.
- It provides integrity that means providing security to the software so that data can be accessed by only an authorized person, no outsider can access the data.

2. Process: It is the foundation layer of software engineering.

- Process defines a framework that includes different activities and tasks.
- The software process covers all the activities and tasks required to be carried out for software development.

3. Method: The third layer establishes the methods of developing the software.

- This includes any technical knowledge required for development.
- It includes collection of tasks starting from communication, requirement analysis, design modeling, program construction, testing, and support.

4. Tools: Software engineering tools provide a self-operating system for processes and methods.

- Tools are integrated which means information created by one tool can be used by another.

[7] explain five activities of a generic process framework in detail

Ans: Software Process Framework is an abstraction of the software development process.

- This article focuses on discussing the Software Process Framework.
- Task sets, umbrella activities, and process framework activities all define the characteristics of the software development process.

- The process framework is required for representing common process activities.

- Five framework activities are described in a process framework for software engineering. Communication, planning, modeling, construction, and deployment are all examples of framework activities.

1. Communication: This framework activity involves heavy communication and collaboration with the customer. It encompasses requirements gathering and other related activities.

2. Planning: This activity establishes a plan for the software engineering work that follows. It describes the technical tasks which are conducted. The resource requires and the work products are produced with a work schedule.

3. Modeling: Architectural models and design to better understand the problem and to work towards the best solution. The software model is prepared by:

- Analysis of requirements
- Design

- It is easy to better understand Software requirements.

4. Construction: Creating code, testing the system, fixing bugs, and confirming that all criteria are met. The software design is mapped into a code by:
   - Code generation
   - Testing

- This activity combines code generation and the testing that is required to uncover errors in the code.

5. Deployment: In this activity, a complete or non-complete product or software is represented to the customers to evaluate and give feedback.
   - On the basis of their feedback, we modify the product for the supply of better products
   - The software delivered to the customer who evaluates the delivered product. It also provides feedback based on the evaluation.

[8] explain smoke testing in detail.
Ans:

Smoke Testing is a software testing method that determines whether the employed build is stable or not.

- Smoke tests are a minimum set of tests run on each build.

- Smoke testing is a process where the software build is deployed to a quality assurance environment and is verified to ensure the stability of the application. Smoke Testing is also known as Confidence Testing or Build Verification Testing.

- Types of Smoke Testing:

There are three types of Smoke Testing:

1. Manual Testing: In this, the tester has to write, modify, or update the test cases for each built product. Either the tester has to write test scripts for existing features or new features.
2. Automated Testing: In this, the tool will handle the testing process by itself providing the relevant tests. It is very helpful when the project should be completed in a limited time.
3. Hybrid Testing: As the name implies, it is the combination of both manual and automated testing. Here, the tester has to write test cases by himself and he can also automate the tests using the tool. It increases the performance of the testing as it combines both manual checking and tools.

- Advantages of Smoke Testing:
- Smoke testing is easy to perform.
- It helps in identifying defects in the early stages.
- It improves the quality of the system.
- It runs quickly.

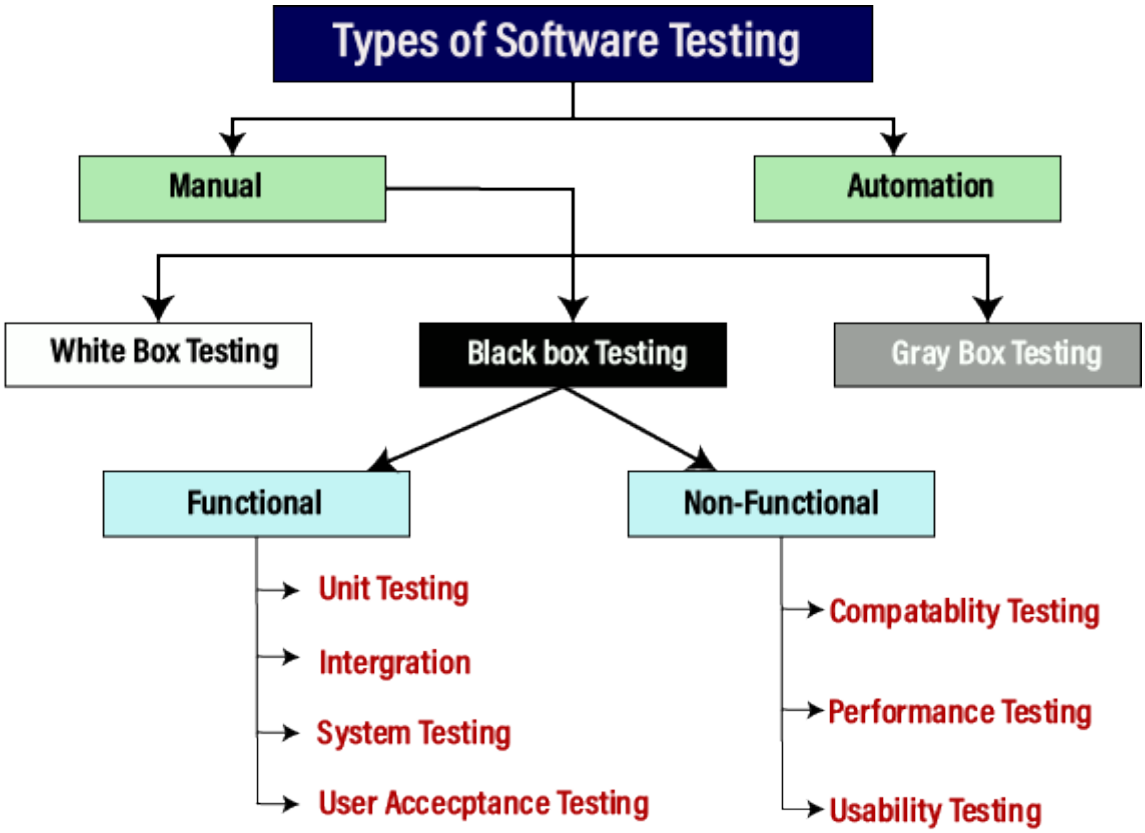
- Disadvantages of Smoke Testing:
- Errors may occur even after implementing all the smoke tests.
- In the case of manual smoke testing, it takes a lot of time to execute the testing process for larger projects.
- It will not be implemented against the negative tests or with the invalid input.

[9] explain Types of Testing.

Ans: testing is required in software engineering because it helps to ensure the quality and functionality of the software product or system.

- Testing is an essential part of SE as it can improve the quality of the software failure.

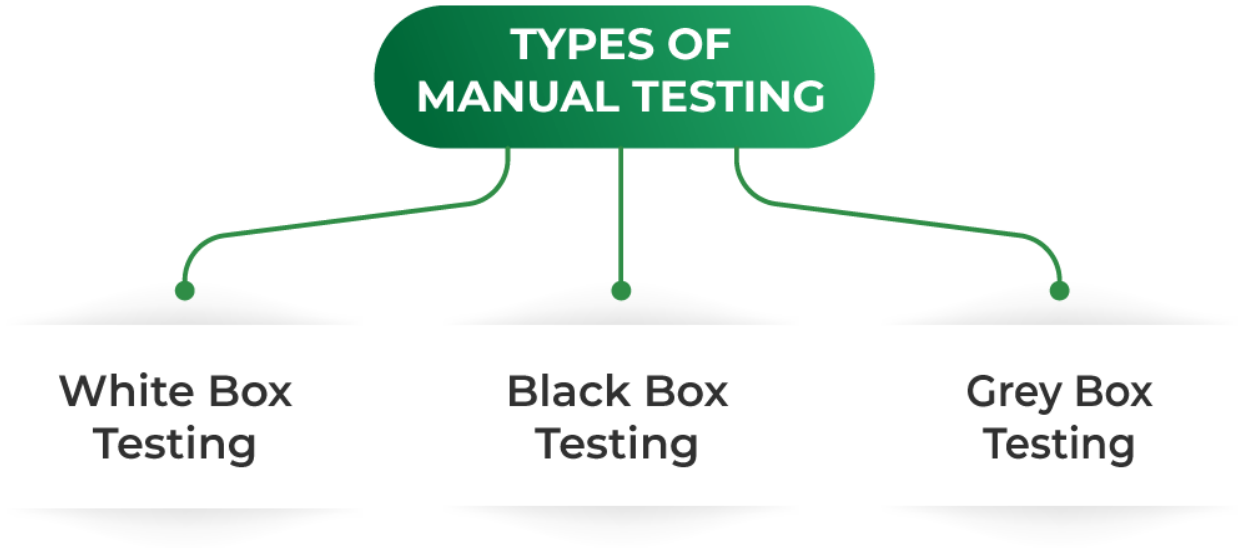
- There are two types of testing:
  1. Manual
  2. Automation.



1. Manual testing: manual testing is a software testing technique that is carried out using the functions and features of an application.

- There are three types:

1. black box
2. white box
3. grey box



1. Black box testing: Black-box testing is a type of software testing in which the tester is not concerned with the internal knowledge or implementation details of the software but focuses on validating the functionality based on the provided specifications or requirements.

- The test engineers perform black box testing.

- To perform black box testing, there is no need have an understanding of the programming language.

- In this, there is no need to know about the internal design of the code.

- Type of Black Box testing:
  The following are the several categories of black box testing:
  1. Functional Testing
  2. Regression Testing
  3. Nonfunctional Testing (NFT)

2. White box testing: white box is such a testing in which the thing you are testing you must know about the internal structure about that.

- The developer can perform white box testing.

- To perform white box testing, we should have an understanding of the programming language
- In this, we will look into the source code and test the logic of the code.
- In this, the developer should know about the internal design of the code.
- White box testing is a software testing technique that involves testing the

internal structure and workings of a software application.

- White box testing is also known as structural testing or code-based testing.
- it is used to test the software's internal logic, flow, and structure.

- Process of White Box Testing

1. Input: Requirements, Functional specifications, design documents, source code.
2. Processing: Performing risk analysis to guide through the entire process.
3. Proper test planning: Designing test cases to cover the entire code. Execute rinse-repeat until error-free software is reached. Also, the results are communicated.
4. Output: Preparing final report of the entire testing process.

3. Grey box testing: Another part of manual testing is Grey box testing. It is a collaboration of black box and white box testing.

Since, the grey box testing includes access to internal coding for designing test cases. Grey box testing is performed by a person who knows coding as well as testing.

In other words, we can say that if a single-person team done both white box and black-box testing, it is considered grey box testing.

2. Automated Testing: Automated Testing is a technique where the Tester writes scripts on their own and uses suitable Software or Automation Tool to test the software.

- It is an Automation Process of a Manual Process.
- It allows for executing repetitive tasks without the intervention of a Manual Tester.
- The goal of automation tests is to reduce the number of test cases to be executed manually but not to eliminate manual testing.

Question: 3

[1] Describe Black Box and White Box Testing in detail.

Ans:

- Black box testing: Black-box testing is a type of software testing in which

the tester is not concerned with the internal knowledge or implementation details of the software but focuses on validating the functionality based on the provided specifications or requirements.

- The test engineers perform black box testing.
- In black box testing, what the software is supposed to do but is not aware of how it does it.
- To perform black box testing, there is no need have an understanding of the programming language.
- In this, we will verify the functionality of the application based on the requirement specification.
- In this, there is no need to know about the internal design of the code.
- It can be applied virtually to every level of software testing.

- Type of Black Box testing:

The following are the several categories of black box testing:

1. Functional Testing
2. Regression Testing
3. Nonfunctional Testing (NFT)

a. Functional Testing: It determines the system's software functional requirements.
b. Regression Testing: It ensures that the newly added code is compatible with the existing code. In other words, a new software update has no impact on the functionality of the software. This is carried out after a system maintenance operation and upgrades.
c. Nonfunctional Testing: Nonfunctional testing is also known as NFT. This testing is not functional testing of software. It focuses on the software's performance, usability, and scalability.

- White box testing: white box is such a testing in which the thing you are testing you must know about the internal structure about that.
- The developer can perform white box testing.
- In white box testing, what the software is supposed to do, also aware of how it does it.
- To perform white box testing, we should have an understanding of the programming language
- In this, we will look into the source code and test the logic of the code.
- In this, the developer should know about the internal design of the code.
- White box testing is a software testing technique that involves testing the internal structure and workings of a software application.
- White box testing is also known as structural testing or code-based testing.
- it is used to test the software's internal logic, flow, and structure.
- It can be applied mainly at unit testing level but now integration, system level also.

- Process of White Box Testing

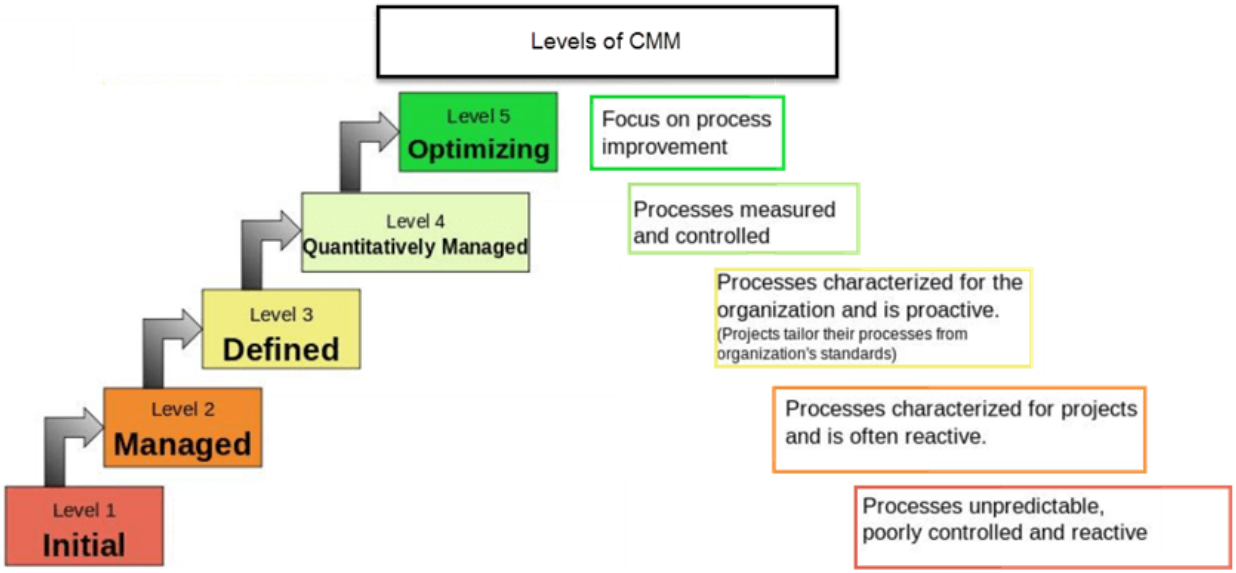
1. Input: Requirements, Functional specifications, design documents, source code.

2. Processing: Performing risk analysis to guide through the entire process.
3. Proper test planning: Designing test cases to cover the entire code. Execute rinse-repeat until error-free software is reached. Also, the results are communicated.
4. Output: Preparing final report of the entire testing process.

[2] What is CMM Level? Explain different level in detail.

Ans: Capability Maturity Model (CMM) was developed by the Software Engineering Institute (SEI) at Carnegie Mellon University in 1987.

- It is a process or methodology used to develop and refine an organization software development process.
- It is not a software process model. It is a framework that is used to analyze the approach and techniques followed by any organization to develop software products.
- It also provides guidelines to enhance further the maturity of the process used to develop those software products.



Levels of Capability Maturity Model (CMM) are as following below.

1. Level One: Initial – Work is performed informally.
A software development organization at this level is characterized by AD HOC activities (organization is not planned in advance.).

2. Level Two: Repeatable – Work is planned and tracked.
This level of software development organization has a basic and consistent project management processes to TRACK COST, SCHEDULE, AND FUNCTIONALITY. The process is in place to repeat the earlier successes on

projects with similar applications.

3. Level Three: Defined – Work is well defined.
   At this level the software process for both management and engineering activities are DEFINED AND DOCUMENTED.

4. Level Four: Managed – Work is quantitatively controlled.
   - Software Quality management – Management can effectively control the software development effort using precise measurements. At this level, organization set a quantitative quality goal for both software process and software maintenance.
   - Quantitative Process Management – At this maturity level, The performance of processes is controlled using statistical and other quantitative techniques, and is quantitatively predictable.

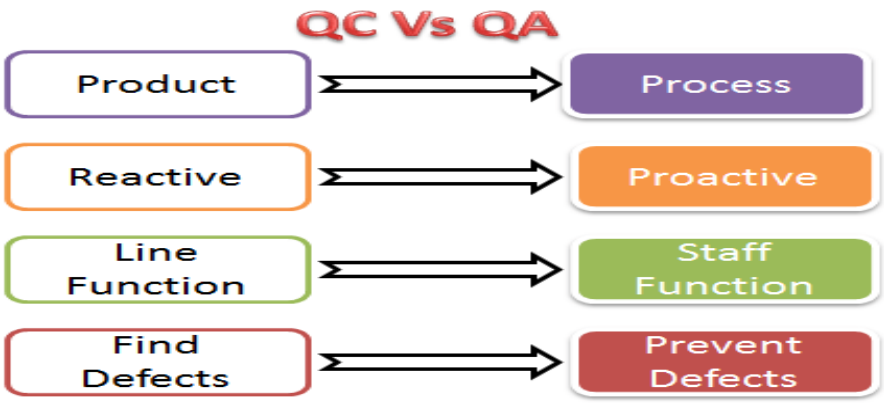
5. Level Five: Optimizing – Work is Based Upon Continuous Improvement.
   The key characteristic of this level is focusing on CONTINUOUSLY IMPROVING PROCESS performance.
   Key features are:
   - Process change management
   - Technology change management
   - Defect prevention

[3] Justify the quality Assurance or Elements of Quality in S.E.

Ans: Software Quality Assurance (SQA) is simply a way to assure quality in the software.

- It is the set of activities which ensure processes, procedures as well as standards are suitable for the project and implemented correctly.
- Software Quality Assurance is a process which works parallel to development of software.
- It focuses on improving the process of development of software so that problems can be prevented before they become a major issue.
- Software Quality Assurance is a kind of Umbrella activity that is applied throughout the software process.
- Purpose of QA is to provide confidence to the customers by constant delivery of product.
- The main aim of SQA is to provide proper visibility of software project.

It reviews the software product and its activities throughout the life cycle of system development



Elements Of Software Quality Assurance:

1. Standards: The IEEE, ISO, and other standards organizations have produced a broad array of software engineering standards and related documents. The job of SQA is to ensure that standards that have been adopted are followed, and all work products conform to them.
2. Reviews and audits: Technical reviews are a quality control activity performed by software engineers for software engineers. Their intent is to uncover errors. Audits are a type of review performed by SQA personnel (people employed in an organization) with the intent of ensuring that quality guidelines are being followed for software engineering work.
3. Testing: Software testing is a quality control function that has one primary goal—to find errors. The job of SQA is to ensure that testing is properly planned and efficiently conducted for primary goal of software.
4. Error/defect collection and analysis: SQA collects and analyzes error and defect data to better understand how errors are introduced and what software engineering activities are best suited to eliminating them.
5. Change management: SQA ensures that adequate change management practices have been instituted.
6. Education: Every software organization wants to improve its software engineering practices. A key contributor to improvement is education of software engineers, their managers, and other stakeholders. The SQA organization takes the lead in software process improvement which is key proponent and sponsor of educational programs.
7. Security management: SQA ensures that appropriate process and technology are used to achieve software security.
8. Safety: SQA may be responsible for assessing the impact of software failure and for initiating those steps required to reduce risk.
9. Risk management: The SQA organization ensures that risk management activities are properly conducted and that risk-related contingency plans have been established.

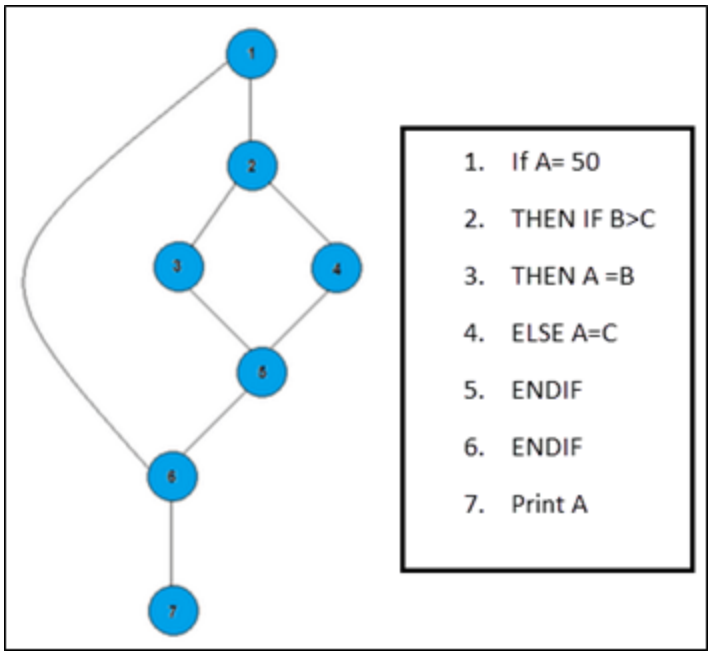
[4] what is Testing. explain basis path testing in detail.

Ans: Testing: Software testing is a procedure of implementing software or the application to identify the defects or bugs.

- Basis Path Testing in software engineering is a White Box Testing method in which test cases are defined based on flows or logical paths that can be taken through the program.
- The objective of basis path testing is to define the number of independent paths, so the number of test cases needed can be defined explicitly to maximize test coverage.

In software engineering, Basis path testing involves execution of all possible blocks in a program and achieves maximum path coverage with the least number of test cases. It is a hybrid method of branch testing and path testing methods.

Here we will take a simple example, to get a better idea what is basis path testing include



- Steps for Basis Path testing

The basic steps involved in basis path testing include

- Draw a control graph (to determine different program paths)
- Calculate Cyclomatic complexity (metrics to determine the number of independent paths)
- Find a basis set of paths
- Generate test cases to exercise each path

- Advantages of Basic Path Testing
  - It helps to reduce the redundant tests
  - It focuses attention on program logic

- It helps facilitates analytical versus arbitrary case design
- Test cases which exercise basis set will execute every statement in a program at least once

[5] what is SCM. explain in detail.

Ans: Software Configuration Management (SCM) is a process to systematically manage, organize, and control the changes in the documents, codes, and other entities during the Software Development Life Cycle.

- The primary goal is to increase productivity with minimal mistakes. SCM is part of cross-disciplinary field of configuration management and it can accurately determine who made which revision.
- The SCM process defines a number of tasks:

1. Configuration Identification: Configuration identification is a method of determining the scope of the software system.
   - With the help of this step, you can manage or control something even if you don't know what it is.
   - It is a description that contains the CSCI type (Computer Software Configuration Item), a project identifier and version information.

Activities during this process:

- Identification of configuration Items like source code modules, test case, and requirements specification.
- Identification of each CSCI in the SCM repository, by using an object-oriented approach
- The process starts with basic objects which are grouped into aggregate objects. Details of what, why, when and by whom changes in the test are made
- Every object has its own features that identify its name that is explicit to all other objects
- List of resources required such as the document, the file, tools, etc.

2. Baseline: A baseline is a formally accepted version of a software configuration item.
   - It is designated and fixed at a specific time while conducting the SCM process.
   - It can only be changed through formal change control procedures.

Activities during this process:

- Facilitate construction of various versions of an application

- Defining and determining mechanisms for managing various versions of these work products
- The functional baseline corresponds to the reviewed system requirements
- Widely used baselines include functional, developmental, and product baselines

In simple words, baseline means ready for release.

3. Change Control: Change control is a procedural method which ensures quality and consistency when changes are made in the configuration object.
   - In this step, the change request is submitted to software configuration manager.

Activities during this process:

- Control ad-hoc change to build stable software development environment. Changes are committed to the repository
- The request will be checked based on the technical merit, possible side effects and overall impact on other configuration objects.
- It manages changes and making configuration items available during the software lifecycle

4. Configuration Status Accounting: Configuration status accounting tracks each release during the SCM process.
   - This stage involves tracking what each version has and the changes that lead to this version.

Activities during this process:

- Keeps a record of all the changes made to the previous baseline to reach a new baseline
- Identify all items to define the software configuration
- Monitor status of change requests
- Complete listing of all changes since the last baseline
- Allows tracking of progress to next baseline
- Allows to check previous releases/versions to be extracted for testing

5. Configuration Audits and Reviews: Software Configuration audits verify that all the software product satisfies the baseline needs. It ensures that what is built is what is delivered.

Activities during this process:

- Configuration auditing is conducted by auditors by checking that defined processes are being followed and ensuring that the SCM goals are satisfied.
- To verify compliance with configuration control standards. auditing and reporting the changes made
- SCM audits also ensure that traceability is maintained during the process.
- Ensures that changes made to a baseline comply with the configuration status

reports

- Validation of completeness and consist

[6] explain Alpha-Beta Testing in detail

Ans:

1. Alpha/beta testing:

a. Alpha testing: any type of testing which is done on developer side is called alpha testing, usually performed with artificial test cases.

- During alpha testing, the product is tested in a testing using white box and black box practices.
- The alpha testing is conducted at developer end.
- It is performed in environment controlled by developer.
- It is performed before software released to end user.
- the developer keeps record of all the errors & problems.
- It involves both black box and white box testing

b. Beta testing: any type of testing which is done at customer side is called beta testing, usually performed with real time data.

- The beta testing is conducted at users place.
- It is performed in environment which is not controlled by developer.
- It is performed after releasing the software to end users.
- The end user record the problems & errors. Later report to developer.
- It involves only black box testing.
- There are two major types of beta testing — traditional and public beta testing

[7] Explain Make / Buy Decision

Ans:

A make-or-buy decision is an act of choosing between manufacturing a product in-house or purchasing it from an external supplier.

[8] Write short note on Risk Refinement

[9] Explain the decomposition for loc base estimation.

Question: 4

[1] Explain difference between ISO and CMM Level

Ans:

ISO	CMM

It is a standards developed by international standard organization	It is a model developed by software engineering institute
Focus is customer supplier	Focus on the software supplier
It is created for hard goods manufacturing industries.	It is created for software industry.
ISO is recognized and accepted in most of the countries.	CMM is used in USA, less widely elsewhere.
This establishes one acceptance level.	It assesses on 5 levels.
Its certification is valid for three years.	It has no limit on certification.
It focuses on inwardly processes.	It focus outwardly.
It has no level.	It has 5 levels
It is basically an audit.	It is basically an appraisal.
It is open to multi sector.	It is open to IT/ITES.
Follow set of standards to make success repeatable.	It emphasizes a process of continuous improvement.

[2] Explain RMMM Plan in detail

Ans: A risk management technique is usually seen in the software Project plan.

- This can be divided into Risk Mitigation, Monitoring, and Management Plan (RMMM).

In this plan, all works are done as part of risk analysis.

- Risk Mitigation:
  - It is an activity used to avoid problems (Risk Avoidance).

  Steps for mitigating the risks as follows.
  1. Finding out the risk.
  2. Removing causes that are the reason for risk creation.
  3. Controlling the corresponding documents from time to time.
  4. Conducting timely reviews to speed up the work.

- Risk Monitoring:
  - It is an activity used for project tracking.

It has the following primary objectives as follows.

1. To check if predicted risks occur or not.
2. To ensure proper application of risk aversion steps defined for risk.
3. To collect data for future risk analysis.
4. To allocate what problems are caused by which risks throughout the project.

- Risk Management and planning:

It is perform after risk have generate in particular project.

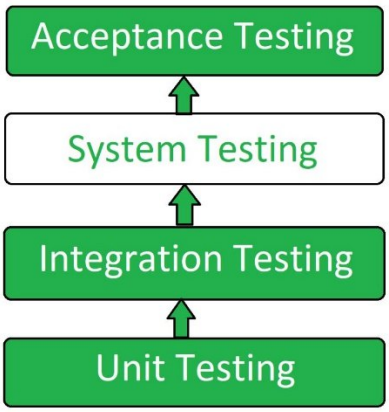
- It assumes that the mitigation activity failed and the risk is a reality. This task is done by Project manager when risk becomes reality and causes severe problems.
- If the project manager effectively uses project mitigation to remove risks successfully then it is easier to manage the risks.
- The main objective of the risk management plan is the risk register. This risk register describes and focuses on the predicted threats to a software project.

- Drawbacks of RMMM:
  - It incurs additional project costs.
  - It takes additional time.
  - For larger projects, implementing an RMMM to be another tedious project.
  - RMMM does not guarantee a risk-free project, infact, risks may also come up after the project is delivered.

[3] What is Category of Training? Explain System Testing in detail.

Ans: System Testing is a type of software testing that is performed on a complete integrated system to evaluate the compliance of the system with the corresponding requirements. In system testing, integration testing passed components are taken as input

- System Testing is carried out on the whole system in the context of either system requirement specifications or functional requirement specifications or in the context of both
- It has both functional and non-functional testing.
- System Testing is a black-box testing.
- System Testing is performed after the integration testing and before the acceptance testing.



Types of System Testing:

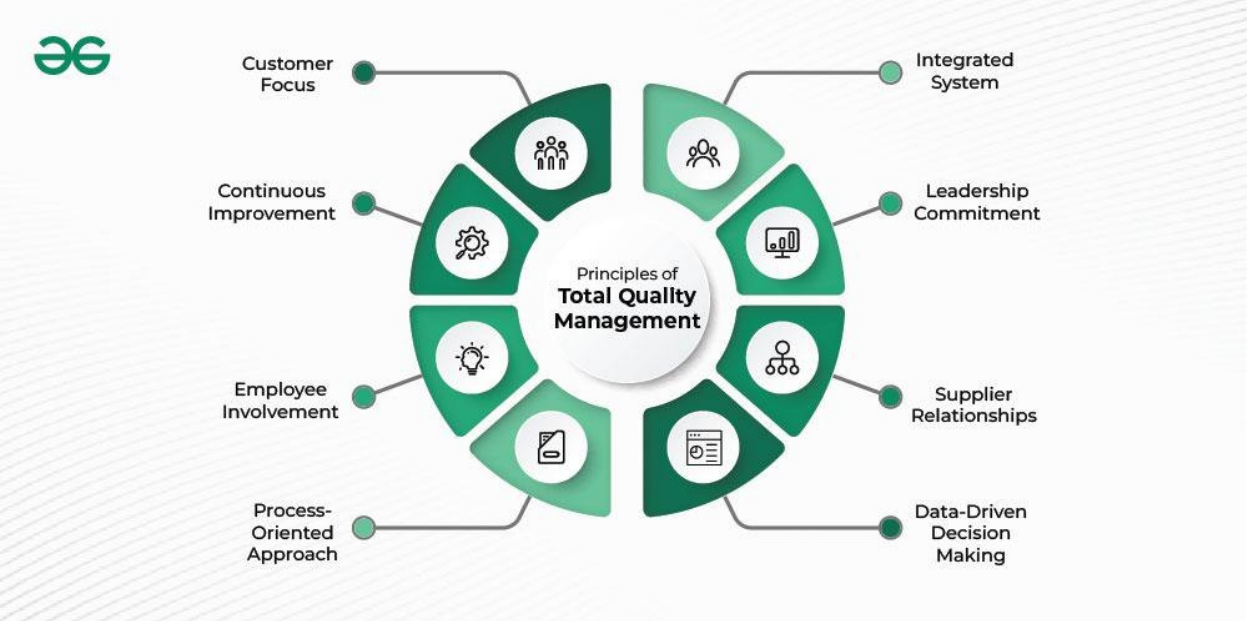
- Performance Testing: Performance Testing is a type of software testing that is carried out to test the speed, scalability, stability and reliability of the software product or application.
- Load Testing: Load Testing is a type of software Testing which is carried out to determine the behavior of a system or software product under extreme load.
- Stress Testing: Stress Testing is a type of software testing performed to check the robustness of the system under the varying loads.
- Scalability Testing: Scalability Testing is a type of software testing which is carried out to check the performance of a software application or system in terms of its capability to scale up or scale down the number of user request load.

[4] explain principle of TQM.

Ans: total quality management is the management activity that integrate all organizational function such as marketing, sales, finance and so on.

- It mainly focus on customer requirements, product quality and top management support.
- Total Quality Management (TQM) is a management philosophy that fosters a culture of excellence, emphasizing continual improvement, customer satisfaction, and active employee involvement

- The TQM made by three word:
  1. Total: the input of every one
  2. Quality: fully meets the customer requirements and customer exception
  3. Management: manage and handle our organization.



- Principles of Total Quality Management (TQM)

The principles of Total Quality Management (TQM) encompass the following:

1. Customer Focus: TQM places a strong emphasis on understanding and meeting customer needs and expectations.

- It involves gathering customer feedback, conducting market research, and using that information to improve products, services, and overall customer satisfaction.

2. Continuous Improvement: TQM promotes a culture of continual improvement throughout the organisation.

- It encourages all employees to actively participate in identifying opportunities for enhancement, eliminating waste, and implementing incremental improvements in processes, products, and services.

3. Employee Involvement: TQM recognizes the importance of involving employees at all levels in quality improvement initiatives.

- It fosters a collaborative and empowered work environment, where employees are encouraged to contribute ideas, make decisions, and take ownership of quality-related activities.

4. Process-Oriented Approach: TQM focuses on managing and improving processes rather than individual tasks or departments.

- It involves mapping, analyzing, and optimizing workflows to enhance efficiency, effectiveness, and consistency.

5. Data-Driven Decision-Making: TQM relies on the collection and analysis of relevant data to support decision-making.

- It emphasizes the use of facts and figures to identify areas for

improvement, measure performance, and monitor progress toward quality objectives.

6. Supplier Relationships: TQM recognizes the significance of strong relationships with suppliers.

- It emphasizes collaboration, communication, and mutually beneficial partnerships with suppliers to ensure the quality of inputs and optimize the overall value chain.

7. Leadership Commitment: TQM requires committed leadership that actively supports and promotes quality principles throughout the organisation.

- Leaders serve as role models, set clear quality goals, provide necessary resources, and foster a culture that prioritizes continuous improvement and customer satisfaction

[5] explain transform mapping and transaction mapping